

9) Travelling Salesman Problem (TSP)

Aim

To develop a Python program to solve the Travelling Salesman Problem (TSP) using the Nearest Neighbor Algorithm.

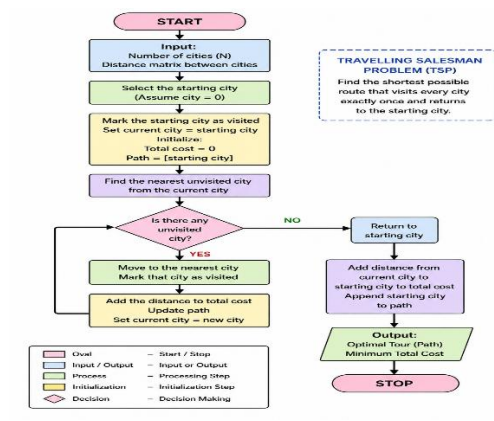
Objectives

- To understand the Travelling Salesman Problem in Artificial Intelligence.
- To find the shortest route that visits every city exactly once.
- To return to the starting city after visiting all cities.
- To implement the TSP using Python.

Algorithm

1. Start the program.
2. Define the distance matrix between cities.
3. Select the starting city.
4. Mark the starting city as visited.
5. Find the nearest unvisited city.
6. Move to the nearest city and mark it as visited.
7. Repeat until all cities are visited.
8. Return to the starting city.
9. Display the tour and total distance.
10. Stop the program.

Flow Chart



Python Program

```
graph = [[0, 10, 15, 20],  
         [10, 0, 35, 25],  
         [15, 35, 0, 30],  
         [20, 25, 30, 0]]
```

```
visited = [False] * 4  
current = 0  
visited[current] = True  
path = [current]  
cost = 0
```

```
for i in range(3):  
    minimum = 999  
    next_city = -1
```

```
    for j in range(4):  
        if not visited[j] and graph[current][j] < minimum:  
            minimum = graph[current][j]  
            next_city = j
```

```
    visited[next_city] = True  
    path.append(next_city)  
    cost += minimum  
    current = next_city
```

```
cost += graph[current][0]  
path.append(0)
```

```
print("Tour :", path)  
print("Minimum Cost :", cost)
```

Sample Output

```
Output  
Tour : [0, 1, 3, 2, 0]  
Minimum Cost : 80  
  
=== Code Execution Successful ===
```

Result

Thus, the Python program to implement the Travelling Salesman Problem (TSP) was executed successfully, and the shortest tour with its total cost was obtained.

Conclusion

The Travelling Salesman Problem (TSP) finds the shortest possible route that visits every city exactly once and returns to the starting city. The Nearest Neighbor Algorithm provides a simple and efficient solution for small datasets and demonstrates an important optimization technique in Artificial Intelligence.

10) TO IMPLEMENT A* ALGORITHM

Aim

To develop a Python program to implement the A* (A-Star) Search Algorithm for finding the shortest path between a start node and a goal node.

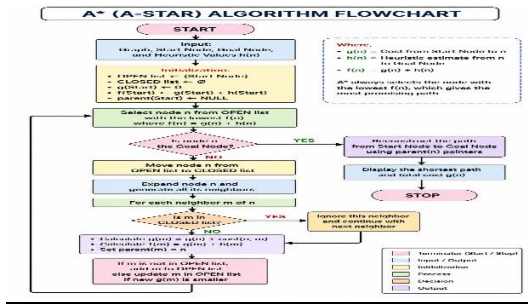
Objectives

- To understand the A* Search Algorithm in Artificial Intelligence.
- To find the shortest path from the start node to the goal node.
- To use heuristic values for efficient path finding.
- To implement the A* algorithm using Python.

Algorithm

1. Start the program.
2. Define the graph and heuristic values.
3. Initialize the open list with the start node.
4. Select the node with the lowest $f(n) = g(n) + h(n)$.
5. If the selected node is the goal node, display the shortest path.
6. Otherwise, expand its neighboring nodes.
7. Update the cost and parent of each neighbor.
8. Repeat until the goal node is reached.
9. Stop the program.

Flow Chart



Python Program

graph = {

'A': [('B', 1), ('C', 3)],

'B': [('D', 3), ('E', 1)],

'C': [('F', 5)],

'D': [],

'E': [('G', 2)],

'F': [('G', 1)],

'G': []

}

heuristic = {

'A': 6,

'B': 4,

'C': 4,

'D': 3,

'E': 2,

```
'F': 1,  
  
'G': 0  
}
```

```
open_list = [('A', 0)]
```

```
visited = []
```

```
while open_list:
```

```
    open_list.sort(key=lambda x: x[1] + heuristic[x[0]])
```

```
    node, cost = open_list.pop(0)
```

```
    if node not in visited:
```

```
        print(node, end=" ")
```

```
        visited.append(node)
```

```
        if node == 'G':
```

```
            break
```

```
        for neighbor, weight in graph[node]:
```

```
            open_list.append((neighbor, cost + weight))
```

Sample Output

```
Output
A B E G
=== Code Execution Successful ===
```

Result

Thus, the Python program to implement the A* Search Algorithm was executed successfully, and the shortest path from the start node to the goal node was obtained.

Conclusion

The A* Search Algorithm combines the actual path cost and heuristic value to efficiently determine the shortest path. It is widely used in Artificial Intelligence, robotics, navigation systems, and game development for optimal path finding.